



Manual técnico

LabSync 1.0 · Arquitectura, instalación, base de datos, pruebas y soporte

Proyecto: Maven multimódulo · Java Swing · JDBC · MySQL/MariaDB.

Punto de entrada: `labsync.aplicacion.AplicacionLabSync`.

Versión documental: 1.0 · **Fecha:** 5 de agosto de 2026.

Audiencia: desarrollo, QA, soporte, administración de base de datos y despliegue.

Control y lectura del documento

Fuente de verdad funcional	Código actual del módulo <code>LabSync</code> , DDL <code>labsync_db/labsync_db.sql</code> , pruebas JUnit y POM vigentes.
Objetivo	Reproducir construcción, instalación, diagnóstico y pruebas; explicar decisiones y puntos de extensión.
Estado verificado	<code>clean verify</code> exitoso con 102 pruebas: 0 fallas, 0 errores, 0 omitidas; JAR sombreado generado.
Evidencia visual	17 capturas renderizadas desde las ventanas Swing reales contra una copia temporal del esquema.
Precaución	Los scripts de instalación y migración deben ejecutarse con respaldo y en el esquema correcto.

Contenido

1. Ficha técnica y estructura
2. Construcción y ejecución
3. Configuración y base de datos
4. Arquitectura del código
5. Flujos y reglas transaccionales
6. Modelo de datos
7. Interfaz, sesión y concurrencia
8. Pruebas automatizadas
9. Pruebas de integración y aceptación
10. Empaquetado y despliegue
11. Operación, respaldo y migraciones
12. Seguridad y deuda técnica
13. Diagnóstico y solución de problemas
14. Checklist de liberación

1. Ficha técnica y estructura

Java 17+nivel objetivo del compilador

61archivos Java principales

102pruebas automatizadas vigentes

14tablas del esquema nuevo

7 intervalo de actualización dinámica

1 JAREjecutable con dependencias

UI	Java Swing, formularios NetBeans, Nimbus	Ventanas por rol, filtros, tablas, diálogos y navegación.
Runtime	JDK 17 o posterior	Compilación y ejecución; se requiere JDK completo, no solo JRE/headless.
Construcción	Maven Wrapper 3.9.11	Reactor multimódulo, pruebas, cobertura, JAR.
Datos	JDBC + MySQL Connector/J 8.3.0	Conexiones, consultas preparadas y transacciones.
Calendario	JCalendar 1.4	Selectores de fecha.
Exportación	Apache POI 5.5.1	Archivos Excel XLSX.
Pruebas	JUnit Jupiter 5.11.4, Mockito 5.15.2	Servicios, persistencia y utilidades de interfaz.
Cobertura	JaCoCo 0.8.13	Reporte HTML/XML durante verify .

1.1 Reactor y rutas clave

LabSync/ ← raíz del repositorio y POM padre |— pom.xml ← packaging

pom; módulo LabSync |— mvnw / mvnw.cmd ← entrada obligatoria de

Maven Wrapper |— LabSync/ | |— pom.xml ← JAR 1.0 y dependencias |

|— src/main/java/labsync/ ← 61 fuentes principales | |—

src/main/resources/ ← imágenes empaquetadas | |— src/test/ ← 10

clases de prueba | |— target/LabSync-1.0.jar ← artefacto

ejecutable |— labsync_db/labsync_db.sql ← instalación completa, 14

tablas y datos demo |— docs/ ← arquitectura, código y diccionario

|— manuales/ ← manuales PDF/HTML y capturas

2. Construcción y ejecución

2.1 Prerrequisitos

- JDK completo 17 o superior: `java` y `javac` deben existir.
- Conectividad a Maven Central en la primera compilación.
- MySQL/MariaDB solo para pruebas de integración o ejecución funcional.
- Servidor gráfico para abrir Swing. En CI sin pantalla, no arrancar el JAR.

```
java -version
javac -version
./mvnw -version
```

Diagnóstico importante: algunas distribuciones instalan un paquete Java *headless* que incluye `java` y el módulo del compilador, pero omite `javac`, `ct.sym` o `libawt_xawt`. En ese caso Maven puede informar “release version 17 not supported” y Swing “HeadlessException”. Instale el paquete JDK/devel completo y el soporte gráfico.

2.2 Construcción reproducible

Desde la raíz, nunca presuponga Maven global:

```
# Linux/macOS
./mvnw clean verify

# Windows PowerShell/CMD
.\mvnw.cmd clean verify
```

El POM padre llama al módulo. La fase `verify` compila con `maven.compiler.release=17`, ejecuta Surefire/JUnit, crea el JAR, lo sombrea con dependencias y genera JaCoCo.

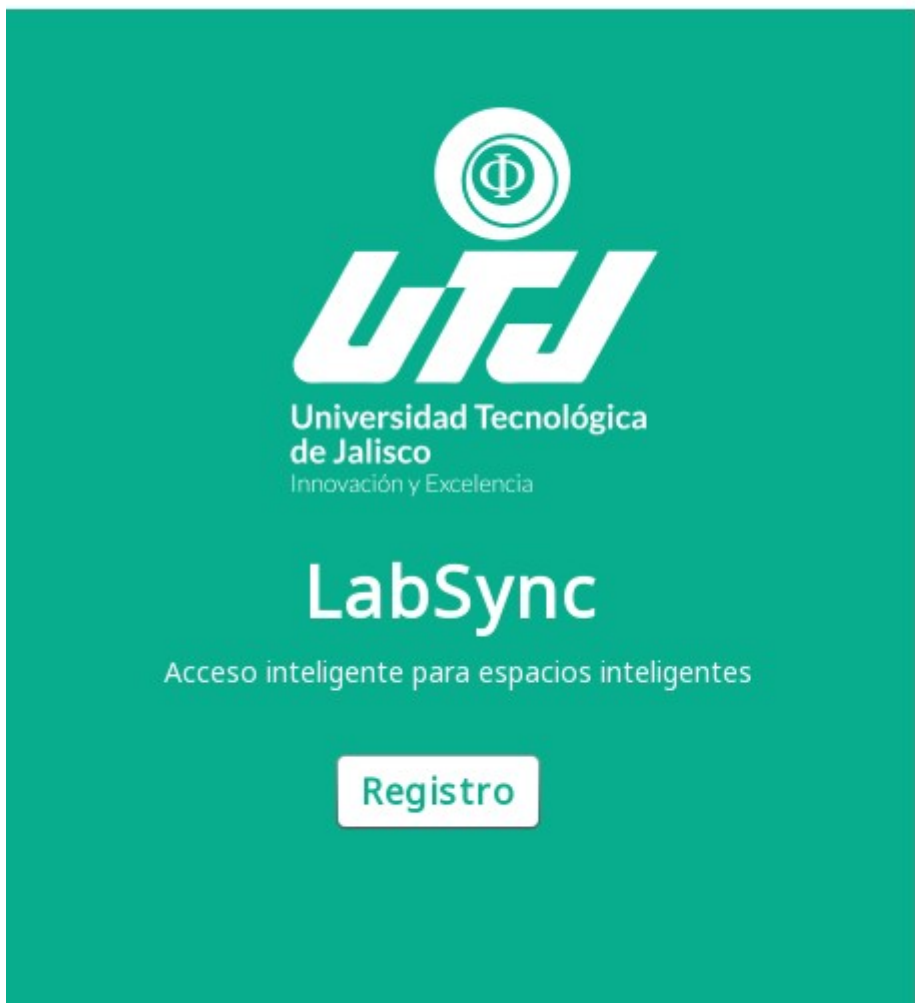
JAR ejecutable	LabSync/target/LabSync-1.0.jar	Manifest contiene <code>Main-Class: labsync.aplicacion.AplicacionLa bSync</code> .
Resultados JUnit	LabSync/target/surefire-reports/	XML/TXT por clase.
Cobertura	LabSync/target/site/jacoco/index.html	Abrir en navegador después de <code>verify</code> .

2.3 Ejecución

```
# Linux/macOS
java -jar ./LabSync/target/LabSync-1.0.jar

# Windows
java -jar .\LabSync\target\LabSync-1.0.jar
```

También puede usarse `Ejecutar-LabSync.cmd`. `AplicacionLabSync.main` delega en `SwingUtilities.invokeLater`, construye `VentanaInicioSesion`, la centra y muestra en el Event Dispatch Thread.



Resultado visual esperado del punto de entrada.

3. Configuración y base de datos

3.1 Precedencia de configuración JDBC

`ConexionBaseDatos` resuelve cada valor en este orden: variable de entorno, propiedad Java y predeterminado.

-D

<code>LABSYNC_DB_URL</code>	<code>labsync.db.url</code>	<code>jdbc:mysql://localhost:3306/labsync_db</code>
<code>LABSYNC_DB_USER</code>	<code>labsync.db.user</code>	<code>root</code>
<code>LABSYNC_DB_PASSWORD</code>	<code>labsync.db.password</code>	cadena vacía

```
# Ejemplo Linux/macOS
export LABSYNC_DB_URL='jdbc:mysql://db-pruebas:3306/labsync_qa'
export LABSYNC_DB_USER='labsync_app'
export LABSYNC_DB_PASSWORD='secreto-administrado-externamente'
java -jar LabSync/target/LabSync-1.0.jar

# Alternativa con propiedades Java
java -Dlabsync.db.url='jdbc:mysql://localhost:3306/labsync_qa' \
  -Dlabsync.db.user='labsync_app' \
  -Dlabsync.db.password='...' \
  -jar LabSync/target/LabSync-1.0.jar
```

No use los predeterminados en producción. Cree un usuario de aplicación con privilegios mínimos; mantenga secretos fuera del repositorio y del historial de comandos.

3.2 Instalación nueva

1. Respalde cualquier esquema homónimo.
2. Inicie MySQL/MariaDB en el host configurado.
3. Importe una sola vez `labsync_db/labsync_db.sql`. El archivo crea/selecciona `labsync_db` y define el esquema completo.
4. Cree datos maestros y cuentas de prueba; el DDL no incluye datos iniciales.
5. Valide exactamente 14 tablas.

```
mysql -u root -p < labsync_db/labsync_db.sql
mysql -u root -p -D labsync_db -e "SHOW TABLES;"
```

Para un nombre de esquema distinto, prepare una copia revisada del DDL y ajuste tanto `CREATE DATABASE/USE` como la URL JDBC. No haga reemplazos a ciegas en producción.

3.3 Migraciones

Instalación completa `labsync_db/`
`labsync_db.sql`

Única fuente SQL: crea directamente 14 tablas y los datos ficticios mínimos.

Procedimiento: respaldo consistente → ensayo en copia → importar `labsync_db/labsync_db.sql` → revisar conteos, restricciones y triggers → ejecutar pruebas de humo → documentar versión. El archivo recrea las tablas, por lo que no debe importarse sobre datos que no estén respaldados.

4. Arquitectura del código

Usuario | eventos Swing ▼ interfaz/* —————> interfaz/comun

(navegación, alertas, actualización) | | |—————>

servicio/* (reglas compartidas y transacciones) | |

—————> persistencia/* (consultas y repositorio de

alertas) | ▼ ConexionBaseDatos —JDBC—> MySQL/MariaDB

4.1 Paquetes

<code>labsync.aplicacion</code>	Punto de entrada <code>AplicacionLabSync</code> .
<code>labsync.configuracion</code>	<code>ConexionBaseDatos</code> , resolución de URL/credenciales y apertura JDBC.
<code>labsync.modelo</code>	Objetos/records de transferencia: <code>SesionUsuario</code> , <code>Alerta</code> , <code>HorarioClase</code> , <code>SoftwareLaboratorio</code> , catálogos.
<code>labsync.persistencia</code>	<code>ConsultaTabla</code> , <code>ConsultaEscalar</code> , <code>CatalogoLaboratorios</code> , <code>RepositorioAlertas</code> .
<code>labsync.servicio</code>	Disponibilidad, horarios, mantenimiento, alertas y software.
<code>labsync.interfaz.autenticacion</code>	Acceso, SHA-256 heredado, alta transaccional de usuario/perfil.
<code>labsync.interfaz.panel</code>	Dashboards por rol.
<code>labsync.interfaz.reservas</code>	Solicitud individual/grupal, historial del profesor y resolución.

<code>labsync.interfaz.bitacora</code>	Registro del profesor y consulta/exportación general.
<code>labsync.interfaz.inventario</code>	Equipos y panel de software por laboratorio.
<code>labsync.interfaz.fallas</code>	Alta por usuario y atención administrativa.
<code>labsync.interfaz.mantenimiento</code>	Programación, estado, selección de filas y exportación.
<code>labsync.interfaz.horarios</code>	Ciclos y asignaciones regulares.
<code>labsync.interfaz.comun</code>	Actualización en segundo plano, modelos, navegación, recursos, validación de fechas, alertas y sonidos.

4.2 Reglas de mantenimiento del GUI

- Los archivos `.form` pertenecen al diseñador NetBeans. No edite manualmente bloques `initComponents` marcados como generados.
- Las consultas lentas no deben ejecutarse en el Event Dispatch Thread. `ActualizacionAutomatica` usa trabajo en segundo plano y aplica el resultado en Swing.
- Toda mutación visual debe volver al EDT.
- Los recursos se cargan desde classpath mediante `Recursos` ; no dependa del directorio actual.
- La navegación preserva `SesionUsuario` y dispone la ventana anterior.

5. Flujos y reglas transaccionales

5.1 Autenticación

correo + contraseña → SHA-256 hexadecimal → SELECT usuario → SesionUsuario → panel por rol

`SesionUsuario` lleva identificador, nombre corto, nombre completo y rol. El ID es esencial para asociar reservas, bitácora y resoluciones al usuario real; no sustituirlo por texto visible.

5.2 Registro

1. Validación de campos, rol y dominio institucional.
2. Captura adicional por rol.
3. `setAutoCommit(false)`.
4. INSERT en `usuario` y lectura de clave generada.
5. INSERT opcional en `estudiante`, `laboratorista` o `externo`.
6. `commit`; ante cualquier excepción, `rollback`.

5.3 Disponibilidad y aprobación

`consultar(laboratorio, fecha, intervalo, rol) ⊢ ¿laboratorio`

`Disponible y capacidad > 0? ⊢ ¿mantenimiento Pendiente/En proceso`

`cuya fecha ya aplica? ⊢ ¿reporte de falla En revisión? ⊢ ¿clase`

`regular activa en ciclo/día con traslape? ⊢ ¿reserva activa de`

`profesor? ⊢ profesor: ¿hay reservas individuales activas? ⊢`

`estudiante: capacidad = total_equipos - alumnos activos ⊢`

`disponible si capacidad restante > 0`

Los estados activos para ocupación son Pendiente y Aprobada. El traslape usa la condición de intervalos `inicioA < finB AND finA > inicioB`. La aprobación reconsulta dentro de transacción y puede añadir `FOR UPDATE` para reducir carreras entre sesiones.

5.4 Horarios regulares

`ServicioHorarios` valida carrera, cuatrimestre 1–11, grupo, turno, materia, módulos de hora y rango del turno. Busca conflictos por laboratorio, profesor o identidad del grupo dentro del ciclo/día. Los triggers repiten integridad crítica en la base.

Matutino	07:00– 14:10	Módulos institucionales de 50 min, con los bloques definidos en el servicio.
Vespertino	15:00– 21:20	Módulos institucionales de 50 min.

5.5 Mantenimiento e inventario

`ServicioMantenimiento` valida el equipo, tipo, fecha, responsable, estado y observaciones especiales; impide conflictos con reservas cuando corresponda; inserta/actualiza en transacción y sincroniza el estado de inventario.

Pendiente o En proceso	Equipo En mantenimiento.
Realizado: preventivo/correctivo/actualización	Disponible y <code>ultimo_mantenimiento</code> actualizado.
Realizado: disposición peligrosa/retiro obsoleto	Baja, sin borrar historial; observaciones obligatorias.
Cancelado	Finaliza la programación; el servicio recalcula el estado aplicable.

5.6 Alertas

`ServicioAlertas` sincroniza avisos idempotentes respaldados por la clave única tipo + referencia y un único origen relacional. Genera alertas para mantenimiento próximo/vencido, preventivo requerido, fallas, equipos con fallas, reservas pendientes y resoluciones.

Los controles de campana y el diálogo actualizan aproximadamente cada 7 segundos. La operación que cambia una alerta debe conservar el origen y actualizar fechas de lectura/atención.

6. Modelo de datos

USUARIO —┐ ESTUDIANTE |┐ LABORATORISTA |┐ EXTERNO |┐

HORARIOS_CLASE — CICLOS_ESCOLARES |┐ RESERVAS —————

LABORATORIOS |┐ BITACORA ————— HORARIOS_CLASE / RESERVAS |┐

REPORTE_FALLAS — INVENTARIO LABORATORIOS —┐ INVENTARIO —

MANTENIMIENTO |┐ SOFTWARE_LABORATORIO |┐ HORARIOS_CLASE |┐

RESERVAS ALERTAS — exactamente un origen: RESERVA | FALLA |

MANTENIMIENTO | INVENTARIO

6.1 Inventario de tablas

usuario	id, correo UK	Identidad, rol y hash heredado.
estudiante	id_usuario	Perfil 1:1; matrícula UK, carrera, turno.
laboratorista	id_usuario	Perfil 1:1; turno y pisos.
externo	id_usuario	Perfil 1:1; institución y motivo.
laboratorios	id_laboratorio, nombre UK	Capacidad de equipos/personas y estado.
ciclos_escolares	id_ciclo, nombre UK	Vigencia de horarios.
horarios_clase	id_horario	Asignación regular; integridad de rol/traslape.

reservas	id_reserva	Solicitud extraordinaria y estados.
bitacora	id_bitacora	Fotografía histórica; referencias opcionales.
inventario	id_inventario, código/serie UK	Equipo, condición y último mantenimiento.
software_laboratorio	id_software, laboratorio+nombre UK	Versiones, uso y acciones pendientes.
reporte_fallas	id_falla	Falla, prioridad, seguimiento y equipo opcional.
mantenimiento	id_mantenimiento	Intervención de inventario y falla opcional.
alertas	id_alerta, tipo+referencia UK	Aviso persistente con un origen exacto.

El detalle campo por campo está en [docs/DICCIONARIO_DATOS.md](#). El DDL usa InnoDB, claves foráneas, índices, CHECK y triggers para reforzar invariantes.

6.2 Reglas de integridad a preservar

- Correos, nombres de laboratorio, matrícula, código/serie y nombres de ciclo no se repiten según sus claves únicas.
- Capacidades y totales de usuarios son positivos.
- Fecha final del ciclo no precede al inicio; hora final es posterior a inicio.
- El profesor de [horarios_clase](#) debe tener rol Profesor.
- La fotografía de laboratorio en bitácora/falla debe coincidir con su FK cuando aplica.
- Una alerta tiene exactamente una FK de origen no nula.
- No elimine físicamente equipos/software que requieran historial: use Baja/Eliminado.

7. Interfaz, sesión y concurrencia

LabSync
TECHNOLOGY

Bitácora

Inventario

Mantenimiento

Ciclos y horarios

Reservas

Reporte de Fallas

UTEJ Universidad Tecnológica de Jalisco
Innovación y Excelencia

Reservaciones para hoy

0

Equipos con

Últimos registros de bitácora

Hora	Usuario	Rol
19:30	Luis Manuel Perez Arce	Profesor
12:06	Luis Manuel Perez Arce	Profesor
21:48	Omar Méndez López	Profesor

Últimos mantenimientos

Fecha	Código Equipo	Nombre Equipo	Lab
31/08/2026	21 25A1 23456	N/A	M-1
05/08/2026	TEST-HW-01	N/A	M-0
17/07/2026	2219A401 753	Lenovo	M-1
16/07/2026	24521	labsync	M-0

El panel administrativo evidencia navegación, sesión, campana y lecturas periódicas.

7.1 Ciclo de vida de ventana

Los constructores reciben preferentemente `SesionUsuario`. Existen sobrecargas con nombre por

compatibilidad, pero el camino nuevo debe conservar el objeto de sesión. Al navegar se crea la siguiente ventana, se centra/muestra y se dispone la anterior. Los temporizadores se detienen al cerrar para evitar fugas.

7.2 Actualización automática

`ActualizacionAutomatica<T>` recibe ventana, intervalo, proveedor de datos y consumidor visual. Un `Timer` inicia lecturas y el resultado se aplica sin bloquear Swing. Varias pantallas usan 7000 ms: dashboards, reservas, fallas, inventario/software, mantenimiento, horarios, bitácora y alertas.

- No cambie el modelo de una tabla si la selección/edición activa se perderá sin restauración.
- Use `ActualizadorModeloTabla` para conservar selección y evitar parpadeo cuando el contenido no cambió.
- Los fallos de refresco deben registrarse y presentar un estado controlado, no cerrar la ventana.

7.3 Exportación

Las ventanas administrativas crean libros XLSX con Apache POI, título, encabezados, filas visibles, formato y autofiltro. Los nombres predeterminados son específicos del módulo. Cierre streams/workbooks con try-with-resources y no ejecute escritura pesada en EDT si el volumen crece.

8. Pruebas automatizadas

8.1 Inventario actual

ActualizadorModeloTablaTest	4	Actualización y selección de tablas.
ModeloAlertasTest	5	Modelo visual de alertas.
NavegacionLaboratoristaTest	2	Rutas del menú.
SeleccionMantenimientoTablaTest	2	Selección y datos de mantenimiento.
RepositorioAlertasTest	17	SQL y estados de alertas.
ServicioAlertasTest	13	Clasificación y mantenimiento requerido.
ServicioDisponibilidadTest	10	Capacidad, bloqueos, traslapes y aprobación.
ServicioHorariosTest	13	Validaciones, conflictos y persistencia.
ServicioMantenimientoTest	18	Tipos, transacciones y efectos de inventario.
ServicioSoftwareLaboratorioTest	8	Validación, alta, filtros y estados.

8.2 Cobertura

El reporte global de la ejecución documentada cubrió 579 de 11,377 líneas (~5.1%) y 282 de 2,177 ramas (~13.0%). El valor global es bajo porque las ventanas Swing generadas son extensas y no están automatizadas en su mayoría; no debe interpretarse como cobertura suficiente del producto. Los servicios críticos tienen concentración mayor.

Criterio de salida: 102 pruebas verdes son necesarias pero no reemplazan los casos funcionales con MySQL/MariaDB, concurrencia, interfaz, exportación y permisos.

8.3 Ejecuciones útiles

```
# Suite completa y cobertura
./mvnw clean verify

# Una clase
./mvnw -pl LabSync -Dtest=ServicioDisponibilidadTest test

# Un método (sintaxis Surefire)
./mvnw -pl LabSync -Dtest=ServicioHorariosTest#nombreDelMetodo test
```

Mockito informa que el auto-attach del mock maker será restringido por futuras versiones de JDK. Trátelo como advertencia de mantenimiento: configure el agente de Mockito cuando se actualice el runtime, sin ocultar indiscriminadamente advertencias.

9. Pruebas de integración y aceptación

9.1 Base aislada

Use un esquema desechable. Una estrategia segura es restaurar un respaldo anonimizado bajo otro nombre, aplicar migraciones y configurar `LABSYNC_DB_URL` a ese esquema. Nunca reutilice producción para casos de transición.

```
CREATE DATABASE labsync_qa CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
-- importar estructura/datos anonimizados
-- apuntar jdbc:mysql://localhost:3306/labsync_qa
```

9.2 Datos mínimos

Usuarios	1 por rol funcional; 2 laboratoristas para concurrencia.
Laboratorios	2 Disponibles con capacidades distintas; 1 No disponible.
Ciclos/horarios	1 ciclo activo, 1 profesor, clases en ambos turnos y un hueco libre.
Reservas	Una por estado y casos traslapados/no traslapados.
Inventario	Disponible, Con falla, En mantenimiento y Baja.
Fallas/ mantenimiento	Una por estado/tipo crítico.
Software	Uno por estado, con duplicado controlado.

9.3 SQL de comprobación

```
-- Tablas y versión funcional
SELECT COUNT(*) AS tablas
FROM information_schema.tables
WHERE table_schema = DATABASE();

-- Reservas por estado
SELECT estado, COUNT(*) FROM reservas GROUP BY estado;

-- Doble ocupación potencial (inspección orientativa)
SELECT a.id_reserva, b.id_reserva, a.id_laboratorio, a.fecha
FROM reservas a JOIN reservas b
ON a.id_reserva < b.id_reserva
```

```
AND a.id_laboratorio = b.id_laboratorio
AND a.fecha = b.fecha
AND a.hora_inicio < b.hora_fin
AND a.hora_fin > b.hora_inicio
WHERE a.estado IN ('Pendiente', 'Aprobada')
      AND b.estado IN ('Pendiente', 'Aprobada');

-- Alertas con origen inválido: debe devolver 0
SELECT COUNT(*) AS invalidas FROM alertas
WHERE ((id_reserva IS NOT NULL) + (id_falla IS NOT NULL) +
      (id_mantenimiento IS NOT NULL) + (id_inventario IS NOT NULL)) <> 1;
```

La consulta de traslapes es orientativa: reservas individuales compatibles pueden coexistir hasta capacidad. Cruce con rol/cantidad antes de reportar defecto.

9.4 Concurrency crítica

1. Abra dos sesiones de laboratorista sobre la misma base.
2. Seleccione la misma reserva Pendiente.
3. Intente aprobar en ambas casi simultáneamente.
4. Verifique un solo resultado coherente, sin pérdida de actualización ni doble ocupación.
5. Repita con dos reservas incompatibles consultadas como libres antes de aprobar.

9.5 Capturas de aceptación



Inicio

Bitácora

Inventario

Mantenimiento

 Ciclos y horarios

Reporte de Fallas

Control de Reservas

Laboratorio

Todos

Fecha	Maestro	Laboratorio	Actividad
2026-08-07	Iván García M...	PB-07	POOas
2026-07-31	Luis Manuel P...	PB-07	POO
2026-07-24	Luis Manuel P...	PB-07	Repaso de la .
2026-07-20	Iván García M...	5-03	Estudio para ..

Ver Detalles

Aprobar

Rechazar

Gestión de reservas usada como evidencia de filtros, tabla y transiciones.



Inicio

Bitácora

Mantenimiento

 Ciclos y horarios

Reservas

Reporte de Fallas

Control de Inventario

Código, marca, modelo o No. serie

Laboratorio

Todos

Equipos

Software por laboratorio

Código	Tipo de Dispos...	Marca	Modelo
TEST-OBS-01	Computadora	Obs	PC obsoleta
TEST-MP-01	Batería de UPS	XD	Batería UPS d
TEST-HW-01	Computadora	Prueba	PC prueba har
2125A123456	Computadora	Lenovo	Prichos
2219A401753	Computadora	Lenovo	15957
24521	Computadora	hp	lenovo
2026A123456	Computadora	hp	1

Registrar Equipo

Editar

Dar de Baja

Inventario y pestaña de software cargados contra el esquema de 14 tablas.



Inicio

Bitácora

Inventario

Mantenimiento

Reservas

Reporte de Fallas



Universidad Tecnológica
de Jalisco
Innovación y Excelencia

Administración de horarios UTJ-CCD

Gestiona cidos y asignaciones regulares de laboratorios

Horarios de laboratorio

Cidos escolares

Ciclo

Todos

Carrera

Todas

Turno

Todos

Día

Todos

Materia, profesor o ID

+ Nueva asignació...

Editar

Activar/deactivar

Ciclo	Carrera	Cuatrimestre	Grupo	Tu
MAYO - AGOSTO 2...	Ingeniería - ITI-ID	7	B	Ve
MAYO - AGOSTO 2...	Ingeniería - ITI-ID	7	B	Ve
MAYO - AGOSTO 2...	Ingeniería - ITI-ID	7	B	Ve
MAYO - AGOSTO 2...	Ingeniería - ITI-ID	7	B	Ve
MAYO - AGOSTO 2...	Ingeniería - ITI-ID	8	B	Ve
MAYO - AGOSTO 2...	Ingeniería - ITI-ID	8	B	Ve

Administración de asignaciones regulares y ciclos.

10. Empaquetado y despliegue

10.1 JAR sombreado

`maven-shade-plugin` reemplaza el artefacto principal e incluye dependencias. `maven-jar-plugin.forceCreation=true` evita volver a sombrar un JAR ya sombreado. El build puede advertir sobre `module-info.class`, licencias/manifestos duplicados y clases de módulo; revise tras actualizar dependencias.

```
jar tf LabSync/target/LabSync-1.0.jar | head
unzip -p LabSync/target/LabSync-1.0.jar META-INF/MANIFEST.MF
```

10.2 Paquete de entrega

- `LabSync-1.0.jar` con hash SHA-256 publicado.
- Script de ejecución correspondiente al sistema.
- Manual de usuario y manual técnico.
- DDL/migración exacta de la versión.
- Notas de versión y matriz de compatibilidad JDK/DB.
- Procedimiento de respaldo y reversión.
- Configuración externa de URL/credenciales.

10.3 Verificación posterior

1. Comparar hash del JAR.
2. Verificar Java, red y esquema de 14 tablas.
3. Iniciar acceso en una estación con pantalla.
4. Ejecutar casos P0 de acceso, consulta, escritura reversible y exportación.
5. Revisar logs de Java y servidor SQL.

11. Operación, respaldo y migraciones

11.1 Respaldo

```
mysqldump --single-transaction --routines --triggers \  
-u operador -p labsync_db > /ruta/fuera-del-repositorio/respaldo.dump
```

Compruebe tamaño, restaure periódicamente en un servidor aislado y proteja el archivo porque contiene correos, nombres y hashes. Defina retención y cifrado según política institucional.

11.2 Monitoreo mínimo

- Disponibilidad y latencia del servidor SQL.
- Crecimiento de bitácora, alertas, fallas y reservas.
- Errores SQL del proceso Java y deadlocks.
- Espacio para respaldos y exportaciones.
- Desfase de reloj entre estaciones y servidor.

11.3 Reversión

El repositorio no ofrece una migración automática de reversa 14 → 13. Antes de migrar, conserve respaldo y binario anterior. Si la liberación falla, detenga escrituras, restaure el respaldo en un esquema nuevo, apunte el binario anterior a ese esquema y valide. No elimine únicamente `software_laboratorio` si la aplicación nueva sigue activa.

12. Seguridad y deuda técnica

Contraseñas con SHA-256 sin sal	Resistencia insuficiente ante robo de hashes.	Migrar gradualmente a Argon2id o bcrypt con sal individual y rehash al iniciar sesión.
Credenciales predeterminadas root/vacía	Privilegios excesivos y configuración insegura.	Usuario dedicado, privilegios mínimos y secreto externo.
JDBC directo sin pool	Conexiones repetidas y observabilidad limitada.	Evaluar datasource/pool y timeouts antes de escalar.
Lógica SQL aún presente en ventanas	Difícil prueba y acoplamiento UI/datos.	Mover gradualmente a servicios/repositorios transaccionales.
Cobertura GUI baja	Regresiones visuales o de navegación.	Pruebas de componentes/robot en entorno gráfico controlado y casos manuales versionados.
Rol Externo sin interfaz	Cuenta válida sin flujo útil.	Ocultar alta, bloquear acceso o implementar el panel según producto.
Mensajes SQL al usuario	Posible exposición de detalles internos.	Registrar detalle técnico y mostrar mensaje neutral/código de incidente.

Las consultas observadas usan `PreparedStatement`, reduciendo inyección SQL. Mantenga esa práctica, valide identificadores dinámicos y nunca concatene entrada del usuario en SQL.

13. Diagnóstico y solución de problemas

<code>release version 17 not supported</code>	Runtime incompleto o <code>ct.sym</code> ausente; <code>java</code> no implica JDK completo.	Instalar JDK/devel; confirmar <code>javac --release 17 -version</code> .
<code>HeadlessException</code>	Sin DISPLAY/servidor gráfico o runtime headless.	No iniciar JAR en CI; en estación, instalar soporte AWT/X11 y usar sesión gráfica.
No hay conexión con la BD	<code>ConexionBaseDatos.conectar()</code> devolvió null.	Revisar URL, DNS/puerto, servicio, usuario, contraseña y permisos.
<code>Table '...software_laboratorio' doesn't exist</code>	Instalación incompleta u obsoleta.	Respalidar e importar <code>labsync_db/labsync_db.sql</code> .
Error de clave duplicada	Correo, matrícula, código, serie, ciclo o software/lab repetido.	Consultar la fila existente; no eludir la restricción.
Reserva rechazada al aprobar	Cambio concurrente, clase, mantenimiento, falla o capacidad.	Es comportamiento esperado si la revalidación explica el bloqueo.
GUI congelada	I/O en EDT, diálogo modal oculto o servidor lento.	Tomar thread dump; buscar AWT-EventQueue y mover consulta a worker.
Modelo de tabla cambia selección	Refresco periódico reemplaza filas.	Usar utilidades de actualización/selección estable.
XLSX no se escribe	Permisos, archivo abierto o sin filas.	Probar ruta escribible, cerrar archivo y revisar excepción POI.

13.1 Recolección de evidencia

```
java -version
./mvnw -version
sha256sum LabSync/target/LabSync-1.0.jar

# Proceso y conexión (adaptar al sistema)
ps -ef | grep '[L]abSync-1.0.jar'
mysql -h host -u usuario -p -D esquema -e 'SELECT 1;'

# Thread dump si la GUI se congela
jcmd PID Thread.print > labsync-thread-dump.txt
```

Adjunte hora exacta, usuario/rol anonimizado, pantalla, acción, identificadores, mensaje completo, logs y consulta de verificación. No adjunte contraseñas ni volcados sin anonimizar.

14. Checklist de liberación

Código y construcción

- JDK completo 17+ confirmado.
- `./mvnw clean verify` exitoso desde raíz.
- 102 pruebas sin fallas, errores u omisiones inesperadas.
- Reporte JaCoCo revisado; regresiones de servicios justificadas.
- JAR ejecutable y manifest correctos.
- Dependencias y advertencias Shade revisadas.

Datos

- Respaldo restaurable disponible.
- Esquema/migración ensayados y 14 tablas verificadas.
- Triggers, constraints e índices presentes.
- Usuario JDBC de privilegios mínimos probado.
- Conteos y consultas de integridad sin anomalías.

Funcional

- Login y navegación de Estudiante, Profesor y Laboratorista.
- Reserva individual/grupal, revalidación y concurrencia.
- Bitácora de clase/reserva sin duplicado.
- Falla → revisión → mantenimiento → cierre.
- Inventario, software, ciclos/horarios y alertas.
- Cinco exportaciones XLSX abiertas y comparadas.
- Cierre de sesión y temporizadores sin procesos residuales.

Entrega

- Hash, notas, manuales y scripts incluidos.
- Plan de reversión aprobado.
- Secretos fuera del paquete.
- Limitaciones (Externo, hash heredado, cobertura GUI) comunicadas.